

SYSTEM DOCUMENTATION

Orca

*A sequential task-execution agent for Google Tasks,
built on Claude Code Routines and Google Cloud.*



Pepper Plant Ventures

Version 1.1 · 21 September 2026

Project YOUR_PROJECT_ID · Region us-east4

1. Overview

Orca turns a Google Tasks list into a work queue for an AI agent. You write a task; Orca picks it up, hands it to Claude, and hands the result back to you for review.

The system runs on a fixed schedule. Every thirty minutes it looks at a single Google Tasks list, takes the topmost task that is ready to run, gathers any supporting material that task needs, and dispatches it to a Claude Code Routine. Claude does the work using whatever tools that routine has been given, and reports back through a push notification. The task stays open until a human reviews the output and ticks it off.

Three properties shape every design decision in the system:

- **Sequential.** One task runs at a time. The queue will not advance while work is in flight, and it will not advance until a person has looked at the result.
- **Human-gated.** Nothing an agent produces goes live on its own. In the reference implementation, campaigns are created as drafts and a person publishes them.
- **Least-privilege.** No downloadable credentials exist anywhere in the system. Each identity can reach only what its job requires, enforced by the platform rather than by instructions in a prompt.

Extending beyond the reference implementation

The first routine built on Orca produces Klaviyo email campaigns, but nothing in the orchestration layer knows anything about email marketing. The orchestrator reads a task, resolves a routine by name, and fires it. What that routine does is entirely defined by its own prompt and its own connectors.

Adding a new class of work means creating a new routine and referencing it by name in a task. The orchestrator requires no code change:

```
Routine: Shopify Product Audit
Prompt:
Review all products tagged "seasonal" and report any with
inventory below 5 units or missing product photography.
```

Plausible extensions include Shopify catalogue maintenance, Figma design handoffs, code changes against a connected repository, and content drafting. Each is a routine definition and a task, not a new system.

DESIGN CONSEQUENCE

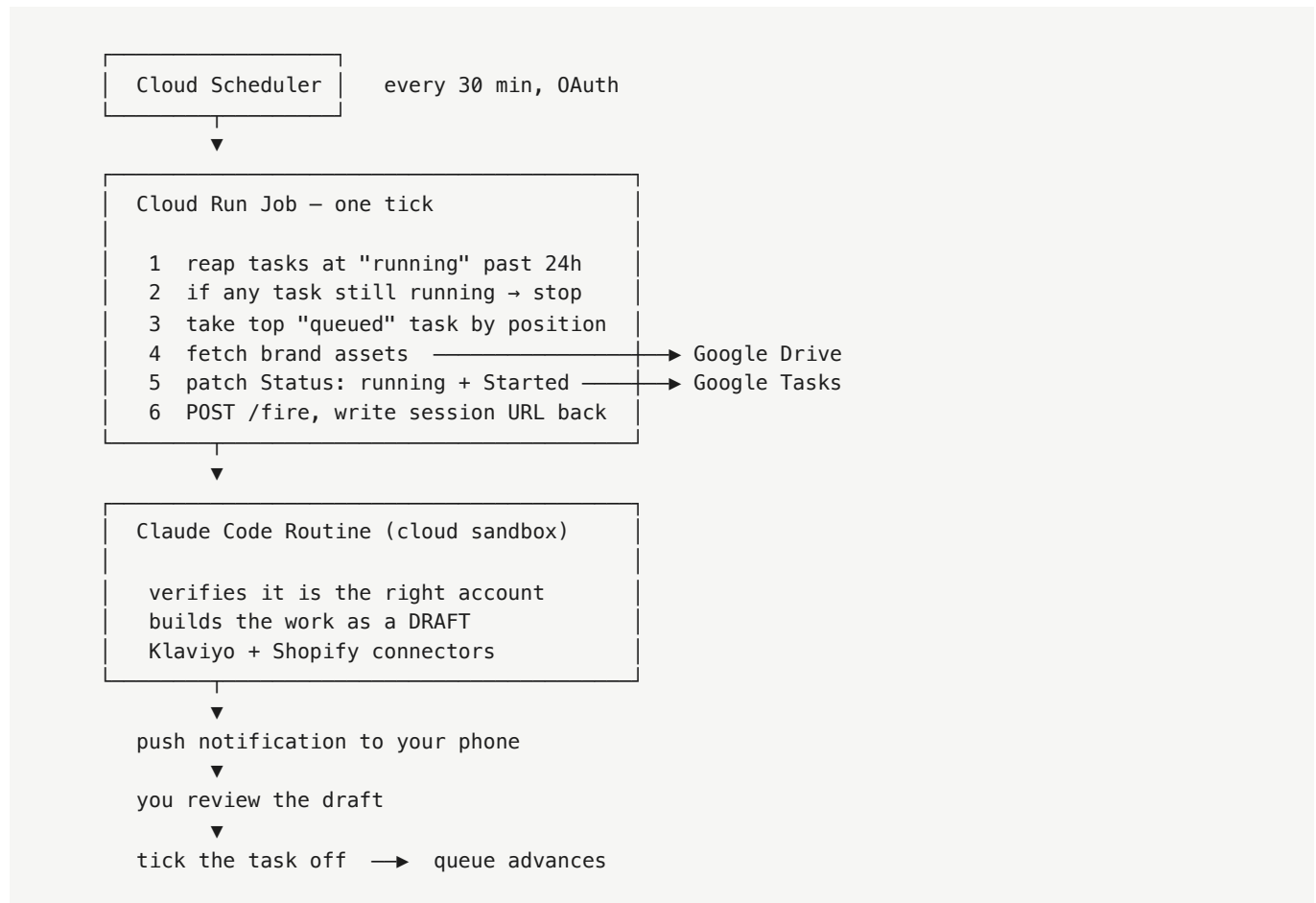
Because routines are addressed by name from free text in a task's notes, the set of routines is effectively a permission surface. A task can only invoke a routine that has been explicitly registered with the orchestrator, and each routine's reach is bounded by the connectors it was given.

2. Architecture

Orca spans four platforms. Each owns one concern, and the boundaries between them are also the security boundaries.

PLATFORM	ROLE IN THE SYSTEM
Google Tasks	The queue and the user interface. Tasks are written and reviewed here; no separate admin surface exists.
Google Drive	Supporting material — brand guidelines, design tokens — read at run time so content can change without redeployment.
Google Cloud Platform	The orchestrator: a scheduled container that reads the queue, resolves assets, and dispatches work.
Claude Code Routines	Execution. A cloud sandbox running Claude with a fixed prompt and a defined set of connectors.

The dispatch cycle



Step 1 is **reaping**: any task that has sat at `running` longer than the timeout is marked `stalled`, on the assumption that the run died or was never reviewed. Without it, one dead task would block the queue forever. Section 3.3 covers the details.

Two decisions worth understanding

STATUS IS PATCHED BEFORE THE ROUTINE FIRES

The routine's fire endpoint has no idempotency key: every request creates a new session. If the orchestrator fired first and then recorded that it had done so, a crash in between would leave a task that looks unstarted — and the next tick would fire it again, producing two real campaigns from one task. Patching first means a crash strands the task instead. A stranded task is recoverable; a duplicate campaign sent to customers is not.

RETRIES ARE DISABLED AT BOTH LAYERS

Cloud Run jobs retry failed tasks by default, and Cloud Scheduler can retry the invocation that starts them. Both are set to zero for the same reason: a retry re-runs a tick that may already have fired. The next scheduled tick is only thirty minutes away, so nothing is lost by waiting.

3. Implementation detail

3.1 Google Tasks — the queue format

Orca reads one task list, by title, defaulting to `Agent Tasks` . A task's **title** is a human label, ignored by the system. All machine-readable state lives in the **notes** field:

```
Routine: Klaviyo Campaign
Status: queued
Started: 2026-09-01T12:30:00+00:00
Session: https://claude.ai/code/session_01GDw...
Prompt:
Create a Klaviyo email campaign based on the concept of a
"Benito's Bundle" – a pre-defined set of three sauces...
```

Parsing splits on the **first** occurrence of `Prompt:` . Everything before it is read as `Key: value` header lines in any order; everything after is the prompt body, verbatim. This matters: an earlier fixed-order regex broke when a prompt body happened to contain the word `Status:` .

FIELD	WRITTEN BY	MEANING
Routine	You	Exact routine name. Resolved to a trigger ID and token by the orchestrator.
Prompt	You	The assignment. Free-form, multi-line.
Status	Orchestrator	Optional. Absent or empty means <code>queued</code> .
Started	Orchestrator	UTC timestamp of dispatch. Used by the reaper.
Session	Orchestrator	Link to the Claude session, for reading the transcript.

STATUS VOCABULARY

VALUE	MEANING
queued	Ready to run. The default when no <code>Status</code> line is present.
running	Dispatched. Blocks the queue until closed.
stalled	Written by the reaper when a task sat at <code>running</code> past the timeout — nobody reported anything.
draft	Written by you to park a half-written task. Any value other than <code>queued</code> keeps a task from firing.

`Status` being optional is deliberate. A hand-written task has no status until the orchestrator stamps one, and requiring the boilerplate line invites the worst failure this system has: a task sitting ignored forever because a line was missed.

The flip side: leaving `Status` blank does *not* hold a task back. Notes autosave as you type and the tick runs every thirty minutes, so a half-written task can fire. Write `Status: draft` **first**, and remove it when the task is ready. A task with no `Prompt:` line cannot be parsed and is skipped whatever its status.

Completion is the native checkbox. There is no `done` status. Ticking the task off in the Google Tasks UI sets it `completed` and `hidden`, which drops it out of the orchestrator's scan entirely and releases the queue. The review you already have to do is the closing action.

Two limits worth knowing: the notes field caps at **8,192 characters**, so link out for anything longer; and ordering follows the Tasks API `position` string, meaning the top of your list is the next thing to run.

3.2 Google Drive — supporting assets

Brand material is read from Drive at dispatch time and injected into the prompt payload. Editing a brand document changes the next run's behaviour with no redeployment.

The access model is the security-critical part. A dedicated service account, `drive-asset-reader`, holds **no domain-wide delegation**. It is a principal in its own right and sees only what has been explicitly shared with its email address, at the `drive.readonly` scope. Two independent limits apply: read-only, and one folder. Neither depends on an agent behaving correctly.

VERIFY AFTER SHARING

Sharing a folder that lives inside a *shared drive* may attach the service account to the entire drive as a member rather than to the single folder. The access is still read-only, so nothing can be modified or deleted — but confirm the scope after sharing rather than assuming it.

WHAT TRAVELS, AND WHAT DOES NOT

Google Docs and Sheets are exported to text; Markdown, JSON and plain text are downloaded directly. Binary assets — logos, photography, PDFs — cannot be inlined into a text payload and are reported separately rather than silently dropped. In the reference implementation these were uploaded once to the Klaviyo image library, and the routine references them from there.

The fetcher also de-duplicates by filename stem. Where the same content exists twice — a `Brand_Voice` Google Doc alongside a `Brand_Voice.md` — it prefers the plain file, which is one direct download and retains its structure, and falls back to the other only if that read fails. Sending both would give the agent two versions of the brand voice and no way to choose.

3.3 Google Cloud Platform — infrastructure

AUTHENTICATION: KEYLESS DOMAIN-WIDE DELEGATION

Reading another user's task list requires domain-wide delegation, which conventionally means downloading a service account JSON key. **No such key exists in this system.** The organisation enforces `constraints/iam.disableServiceAccountKeyCreation`, and that policy was deliberately kept rather than carved out — an exception would re-enable downloadable keys for every service account in the project permanently, and would not revoke any key already created.

Instead, Google signs the delegation assertion on request:

1. Application Default Credentials resolve the caller — a developer locally, the attached service account in Cloud Run. Same code path either way.
2. That identity calls IAM Credentials `signJwt`, asking Google to sign a claim set with the service account's Google-managed private key, which is never exposed.
3. The signed JWT is exchanged at the OAuth token endpoint for an access token scoped to the impersonated user. The `sub` claim is what performs the delegation.

RESOURCES

RESOURCE	PURPOSE
agent-tasks-orchestrator	Service account. Delegated access to Tasks; can sign as itself and impersonate the Drive reader.
drive-asset-reader	Service account. No delegation; read-only, folder-scoped Drive access.
orca-orchestrator	Cloud Run job — one tick per execution, no web server, retries disabled.
orca-tick	Cloud Scheduler job. Invokes the above by OAuth, retries disabled.
routine-token-*	Secret Manager. One secret per routine, mounted as an environment variable at run time.

All of the above is defined in Terraform under `infra/`, including API enablement and every IAM binding. There is no public endpoint anywhere in the deployment.

THE REAPER AND THE SEQUENTIAL GATE

Every tick begins by reaping. A task at `running` for longer than `REAPER_TIMEOUT_SECONDS` — default **24 hours** — is marked `stalled`. A task at `running` with no usable timestamp is flagged for a human and never reaped automatically, because reaping on no evidence could kill a live run.

The timeout covers *review* latency, not run time. A task leaves `running` when a person ticks it off, so reaping at, say, four hours would release the gate overnight and fire the next campaign before anyone had seen the last one. The cost of the longer window is that a genuinely dead run blocks the queue for a day; at a handful of tasks per week, that is the right trade.

After reaping, if anything is still running, the tick fires nothing. Reaping before checking is what stops a dead task from blocking the queue permanently.

3.4 Claude Code Routines — execution

A routine is a named, reusable execution context: a saved prompt, a model, a set of MCP connectors, and an API trigger. The orchestrator holds each routine's trigger ID and bearer token, and dispatches by POST to its fire endpoint with an `anthropic-version` header.

PAYLOAD STRUCTURE

```
BRAND ASSETS
--- Brand_Kit.json ---
{ "color": { "green": { "hex": "#039700", ... } } }
--- Brand_System.md ---
...

TASK
Create a Klaviyo email campaign based on...
```

This arrives inside a `<routine-fire-payload>` wrapper which instructs the agent to treat the contents as **data, not instructions**, unless the routine's own prompt opts in. **The routine prompt must contain that opt-in**, or fired tasks arrive as inert context and the run does nothing. The payload caps at 65,536 characters; the reference implementation uses roughly 22,000.

ANATOMY OF A ROUTINE PROMPT

The saved prompt is where operating limits are expressed. The reference routine covers, in order: what the agent is; that the fire payload is its assignment; where brand assets come from and that Drive is unavailable; that imagery comes from the Klaviyo library and products from Shopify; a tripwire to halt if a connector resolves to the wrong account; a prohibition on inventing products, prices or image URLs; the instruction to build a **draft** and never send, schedule or publish; that a human closes the task and the agent must not attempt to; and a closing instruction to report what was created and what was assumed.

PROMPTS ARE NOT A SECURITY BOUNDARY

Every instruction above is a request the model is very likely to honour, not a constraint it cannot violate. Real limits come from what the routine can reach: the connectors attached to it, and the scopes those connectors hold. When something genuinely must not happen, remove the capability rather than forbidding its use. This is why Drive was removed from the routine entirely and replaced with a pre-fetch through a read-only, folder-scoped account.

CONNECTORS

Connectors are account-level MCP integrations attached per routine — Klaviyo and Shopify in the reference implementation. They inherit the permissions of the connected account, so an agent holding a Shopify connector has whatever access that connection grants, including writes. Connector scope is therefore a deployment decision, not a prompt-writing one.

Routines run in a cloud sandbox with a default network allowlist. Reaching any host outside it requires adding the domain under the environment's custom network access settings.

4. Deploying for a client

Deployment is scripted. `./deploy.sh` is interactive and safe to re-run; every step is idempotent and Terraform owns the state.

Before running the script

Have these in place first. The script checks most of them and pauses if one is missing, but it cannot create any of them:

- **Tools:** `gcloud`, Terraform 1.5 or later, and `python3`. Docker is not needed; Cloud Build builds the image.
- **Two logins, one account:** `gcloud auth login` and `gcloud auth application-default login`.
- **A GCP project** with billing linked and `roles/owner` (or the narrower set the script prints) for the deploying account.
- **The routines, created in Claude first.** The script prompts for each routine's `trig_...` ID and its `sk-ant-oat` token, which come from the routine's API trigger.

What the script checks before touching anything

Each check corresponds to a failure that occurred during the original build, and each one misleads if you meet it cold.

CHECK	WHY IT EXISTS
Active account	<code>gcloud</code> stays authenticated as whoever you last logged in as. Switching to an admin account to fix billing and forgetting to switch back produces <code>PERMISSION_DENIED</code> on a project you own — which looks exactly like an org policy problem and is not.
Terraform credentials	Terraform authenticates through Application Default Credentials, a separate login from the <code>gcloud</code> CLI. If those are missing or belong to another account, every other check passes and <code>terraform apply</code> then fails with <code>PERMISSION_DENIED</code> . The script compares the two and offers to redo the ADC login.
Permissions	Tests the seven permissions actually required. If any are missing it prints the roles and the exact command, then pauses and re-checks so the work can be handed to an administrator without restarting.
Billing	Secret Manager, Cloud Run and Scheduler all refuse to enable without it.
Org policy	Detects a service-account-key restriction and reports it as <i>compatible</i> — this design mints no keys.

Billing: the trap worth knowing in advance

Linking a billing account needs **two permissions held by one identity**, on two different resources:

- On the **project**: `roles/owner` or `roles/billing.projectManager`
- On the **billing account**: `roles/billing.user` or `roles/billing.admin`

Billing account roles are granted on the billing account's own permissions page, not on the project's IAM page — which is why the role you need does not appear when you search for it in the wrong place. A Google Workspace super administrator holds **neither** of these by default; Workspace admin and GCP IAM are separate systems.

If linking reports `Cloud billing quota exceeded`, the billing account has hit its project cap. Unlink a dead project, or request an increase.

Steps Terraform cannot perform

The deploy prints these on completion with the correct values substituted. The system will not function until all are done.

1. **Authorise domain-wide delegation.** In the Workspace Admin console under Security → Access and data control → API controls → Domain-wide delegation, add the orchestrator's **numeric client ID** — not its email — with scope `https://www.googleapis.com/auth/tasks`. Requires a super administrator.

FAILURE SIGNATURE

Until this is authorised, every step succeeds up to the final token exchange, which fails with `unauthorized_client`. Signing worked; delegation is what is missing.

2. **Share the assets folder** with the Drive reader service account as Viewer, then verify the resulting scope.
3. **Create the task list** under the impersonated user's account, matching the configured title.
4. **Check the routines** in Claude Code. They already exist, since the script needed their trigger IDs; confirm each prompt opts in to the fire payload.

Configuration

Routines are configured as a map, so a client with three routines adds three entries. Naming is derived automatically: `"Klaviyo Campaign"` becomes the environment variable `ROUTINE_ID_KLAVIYO_CAMPAIGN` and the secret `routine-token-klaviyo-campaign`. Tokens are prompted for without echo, written under a restrictive umask, and excluded from version control. A validation rule rejects tokens that do not begin with `sk-ant-oat`, because pasting an API key instead of a routine token fails obscurely much later.

Adopting an existing deployment

A project built by hand before this configuration existed can be adopted with `infra/import.sh`, which imports each resource into Terraform state and is safe to re-run. A fresh client deployment should not use it.

Per-client checklist

ITEM	NOTES
GCP project with billing	Verify both halves of the billing permission before starting.
Workspace super admin access	Required once, for domain-wide delegation.
Claude account with Routines	Found under More → Routines in the desktop app; labelled <i>Scheduled</i> in the web sidebar.
Connectors authorised	Connected at account level before attaching to a routine.
Assets folder	Optional. Read-only share to the Drive reader.
Tick interval	Thirty minutes suits review-gated work. The sequential gate means a shorter interval increases responsiveness, never concurrency.

Operating notes

- **A task stuck at `running`** after a crash: `python orchestrator.py --reset <task_id>`. The reaper handles it automatically after the timeout.
- **Previewing a `dispatch`** without firing: `--dry-run` prints the exact payload and touches nothing.
- **Reaping on its own schedule:** `--reap-only`.
- **Reading a run:** the session URL is written onto the task; the full transcript is available there.

A NOTE ON SCOPE

This system is deliberately small. It has no database, no web interface, no user accounts and no public endpoints. The queue is a Google Tasks list, the state is five lines of text in a notes field, and the audit trail is the session transcript. That is enough for review-gated work at a handful of tasks per week, and each of those omissions removes a category of failure. Scale it up only when something concrete demands it.